Git Guide.

Every Git and GitHub answer in one place.

Ask in plain language, or paste the error Git printed.
Exact commands, a danger level for each, and its undo.

1000 answers · **500** errors decoded · **23** lessons you can run

an undo for everything · works offline · no trackers · MIT

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

A note from Amey

Git carries nearly every codebase on earth, and almost everyone learned it by accident: a command from a teammate, a midnight search, a ritual repeated without understanding.

The gap was never which command to type. It is the question nobody answers first: **will this destroy my work, and can I get it back**. Documentation says what a command does, rarely what it costs.

So every answer here carries two things. A danger level, in three words: **safe**, **rewrites history**, **can lose work**. And its undo, or a plain statement that none exists.

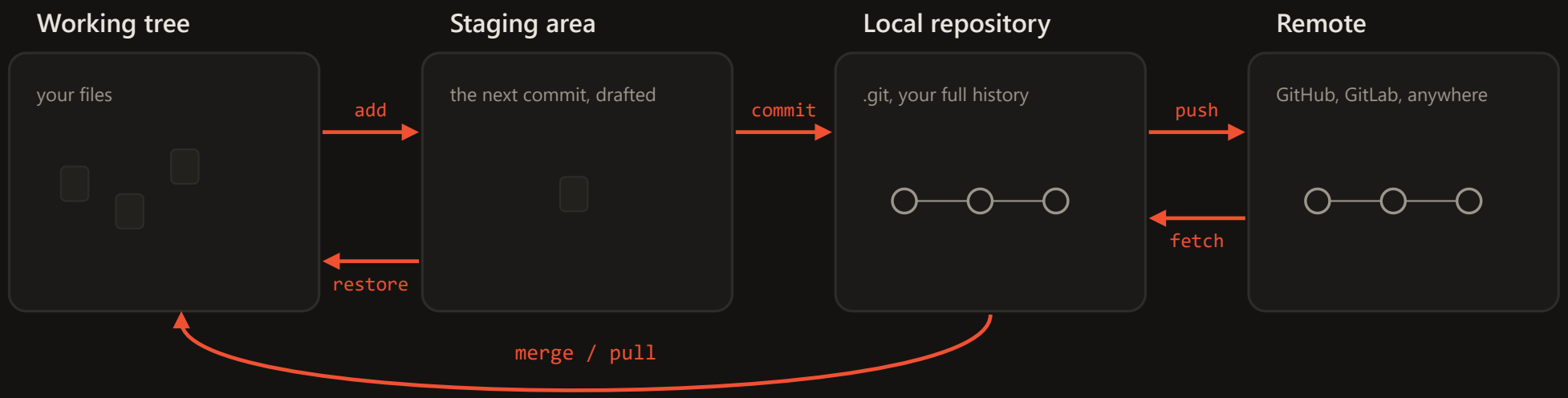
Read these in any order. The model comes first because everything else follows from it, and the errors near the back are worth a skim now, so meeting one later feels like recognition rather than panic.

This is the portable half. The other half is at **amey-thakur.github.io/GIT-GUIDE**: a search box over the same 1000 answers and 500 decoded errors, and a Git engine you can break on purpose and put back.

That last one is the point. **Committed work is almost always recoverable. Uncommitted work is not.** Believe those two sentences and Git stops being something to be careful around.

Amey

Where your code lives



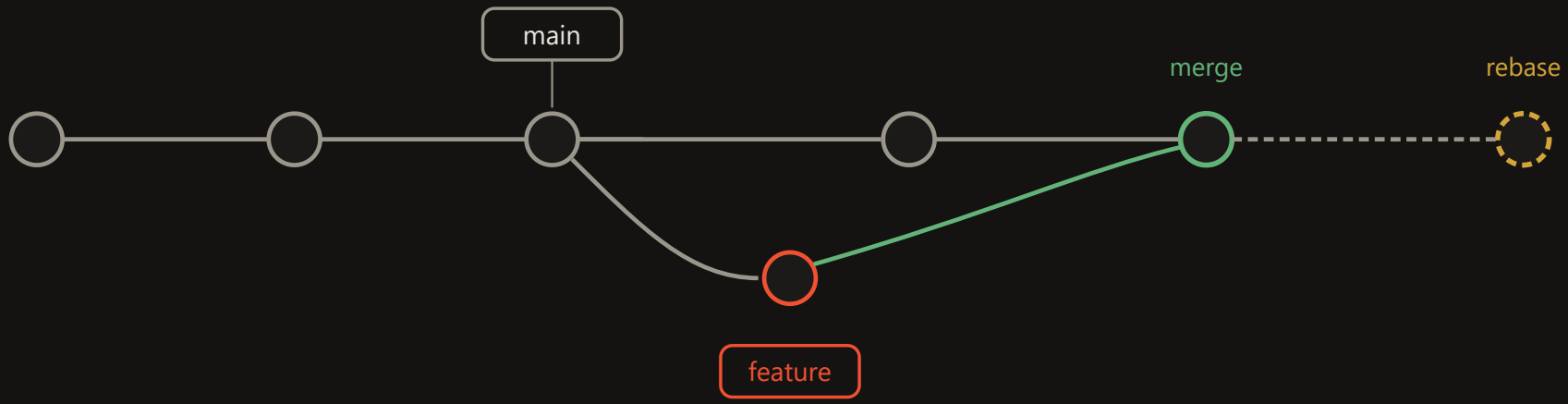
add copies a change into the staging area, the draft of your next commit. You choose what goes in.

push uploads your commits; **fetch** downloads theirs and stops. **pull** is fetch plus merge.

commit seals the draft into a permanent snapshot in your local repository. Still only on your machine.

restore copies recorded versions back over your files; **reset** moves the branch pointer itself. That is where history is un-happened.

Commits, branches, HEAD



A **branch** is a movable label on one commit. Creating one copies nothing; it is instant and free.

A **rebase** replays your commits as new ones for a straight line. It rewrites history: your own unshared work only.

A **merge** commit has two parents and joins the lines. History shows what truly happened; always safe on shared branches.

HEAD is where you stand, normally attached to a branch. Standing on a commit directly is detached HEAD: a state, not an error.

From zero to first push

1 Install

Windows: winget install Git.Git · macOS: brew install git · Debian: sudo apt install git

```
git --version
```

2 Identity

The same email as your GitHub account, or commits will not count on your profile

```
git config --global user.name "<name>" && git config --global user.email "<email>"
```

3 One key, every host

Add the public key to GitHub, GitLab, Bitbucket, Azure: the same key works on all

```
ssh-keygen -t ed25519 -C "<email>"
```

4 First repository

Joining: git clone <url> · Publishing a folder of yours:

```
gh repo create <name> --public --source . --push
```

5 The loop you will run forever

Edit, stage, commit, push. Everything else is a variation

```
git add <file> && git commit -m "<message>" && git push
```

6 Keep the noise out

Ignore build output and dependencies from the first commit, not the tenth

```
curl -o .gitignore https://raw.githubusercontent.com/github/gitignore/main/Node.gitignore
```

7 When it goes wrong

It will, for everyone. Ask the site in plain language, or paste the exact error

```
amey-thakur.github.io/GIT-GUIDE
```

Amey Thakur

amey-thakur.github.io/GIT-GUIDE

Branches

Cheap, fast, and safe to make by the dozen.

`git switch -c <branch>`
Create and move in one step

`git switch <branch>`
Move to an existing branch

`git switch -`
Back to the branch you just left

`git branch --show-current`
Where am I

`git branch -vv`
Every local branch with its remote and ahead-behind state

`git branch -a`
Every branch, local and remote

`git push -u origin <branch>`
Publish a new branch

`git branch -d <branch>`
Delete a merged branch; -D forces

`git push origin --delete <branch>`
Delete it on the remote too

`git branch -m <old> <new>`
Rename

Inspect and search

Read history like a book with an index.

`git log --oneline --graph --all`
The whole history as a graph

`git log -p -- <file>`
Every change to one file; --follow crosses renames

`git log --grep="<text>"`
Find commits by message

`git log -S "<text>"`
Find commits that added or removed this text

`git diff main...<branch>`
What the branch changed since it split; the pull request view

`git shortlog -sn --all`
Commits per author, ranked

`git blame -w <file>`
Who last touched each line, ignoring whitespace

`git show <hash>`
One commit in full; --stat for files only

`git bisect start`
Binary-search history for the commit that broke it

`git branch -a --contains <hash>`
Which branches carry this commit

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

Setup and identity

Once per machine; everything else builds on it.

`git config --global user.name "<name>"`
Name stamped into your commits

`git config --global user.email "<email>"`
Use your host account email so commits link to your profile

`git config --global init.defaultBranch main`
New repositories start on main

`git config --global core.editor "code --wait"`
Commit messages open in VS Code

`git config --global pull.ff only`
Pull never creates surprise merge commits

`git config --global fetch.prune true`
Deleted remote branches disappear locally too

`git config --global rebase.autoStash true`
Pull with rebase works even with edits in progress

`git config --global core.autocrlf true`
Windows line endings handled once; macOS and Linux use input

`git config --list --show-origin`
Every setting and the file it comes from

The daily loop

The nine commands that are most of Git.

`git status -sb`
What changed, one line per file

`git diff`
Unstaged edits, line by line

`git diff --staged`
What the next commit will contain

`git add <file>`
Stage a change; git add -p picks piece by piece

`git commit -m "<message>"`
Seal the staged snapshot

`git commit --amend --no-edit`
Add staged changes to the last commit

`git fetch`
See what the remote has before touching anything

`git pull --rebase`
Get remote commits, replay yours on top

`git push`
Publish your commits

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

Undo and rescue

Every mistake has a way back.

`git restore <file>`
Discard edits, back to last commit

`git restore --staged <file>`
Unstage, keep the edits

`git commit --amend -m "<new message>"`
Rewrite the last commit message

`git reset --soft HEAD~1`
Uncommit, keep everything staged

`git reset --hard HEAD~1`
Erase the last commit and its work

`git revert <hash>`
Undo by adding an opposite commit; safe when pushed

`git reflog`
Every state you have been in; the safety net

`git reset --hard ORIG_HEAD`
Undo the merge, rebase, or pull that just happened

`git clean -nd`
Preview deleting untracked files; -fd deletes

Every platform, same Git

Git is identical everywhere; only the remote URL and sign-in differ. The same SSH public key works on every host.

`git clone git@github.com:<user>/<repo>.git`
GitHub over SSH

`git clone git@gitlab.com:<user>/<repo>.git`
GitLab

`git clone git@bitbucket.org:<workspace>/<repo>.git`
Bitbucket

`git clone https://dev.azure.com/<org>/<project>/_git/<repo>`
Azure DevOps (Azure Repos)

`git clone https://git-codecommit.<region>.amazonaws.com/v1/repos/<repo>`
AWS CodeCommit

`git config --global credential.helper manager`
Browser sign-in for HTTPS remotes, all hosts

`git remote set-url origin <url>`
Move between hosts or between HTTPS and SSH

`git clone --mirror <old> && git push --mirror <new>`
Migrate anywhere with full history

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

Merge and rebase

Two ways to bring lines together.

`git merge <branch>`
Bring a branch in; history stays true

`git merge --no-ff <branch>`
Always keep the feature visible as one bubble

`git rebase main`
Replay your branch on top of main

`git rebase -i HEAD~<n>`
Rewrite, reorder, squash your last n commits

`git cherry-pick <hash>`
Copy one commit onto this branch

`git merge --abort`
Back out of a conflicted merge

`git rebase --continue`
Resume after resolving a conflict

Quality of life

Small settings, daily comfort.

`git config --global help.autocorrect prompt`
git statsu asks: run git status instead?

`git config --global alias.st "status -sb"`
git st forever

`git config --global alias.lg "log --oneline --graph --all"`
git lg: the graph

`git config --global rerere.enabled true`
Never resolve the same conflict twice

`git config --global gpg.format ssh`
First of three lines to signed, Verified commits

`git commit --allow-empty -m "rerun"`
Retrigger CI without touching code

`git config core.hooksPath .githooks`
Versioned hooks the whole team shares

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

No host at all

Git predates the hosts and needs none of them.

`git init --bare ~/server/repo.git`
Your own remote on any machine or shared drive

`git bundle create repo.bundle --all`
The whole repository as one file: USB, email, air gap

`git clone repo.bundle -b main <folder>`
Clone from the file, no network

`git format-patch -3`
Last three commits as mailable patch files

`git am <file>.patch`
Apply mailed patches, authorship intact

`git archive --format=zip -o src.zip HEAD`
Clean source export, no .git

Files and ignoring

What Git tracks, and what it must never.

`printf "node_modules/\n" >> .gitignore`
Ignore by pattern; affects untracked files only

`git rm --cached <file>`
Stop tracking, keep on disk

`git mv <old> <new>`
Rename and stage in one step

`git update-index --skip-worktree <file>`
Ignore your local edits to a tracked file

`printf "* text=auto\n" >> .gitattributes`
Line endings settled for the whole team, committed once

`touch <folder>/.gitkeep`
Keep an empty folder

Worth knowing

A repository in one file
`git bundle create repo.bundle --all`
Clone straight from it. Crosses air gaps and email.

Any folder can be a remote
`git init --bare /path/repo.git`
No server, no account, still a real remote.

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

Scale and speed

Big repositories, kept quick.

`git clone --filter=blob:none --sparse <url>`
Huge repository, download almost nothing

`git sparse-checkout set <folders>`
Materialize only what you work on

`git worktree add ../<repo>-review <branch>`
Second folder, second branch, zero stashing

`git lfs track "*.psd"`
Large binaries stored properly

`git count-objects -vH`
How heavy is this repository really

`git maintenance start`
Background upkeep keeps big repositories fast

Start a project

From nothing, or from a URL.

`git init -b main`
Turn this folder into a repository

`git clone <url>`
Copy a repository, any host, full history

`git clone --depth 1 <url>`
Latest snapshot only, fastest

`git remote add origin <url>`
Connect a local repository to a host

`git push -u origin main`
First push; -u links the branches for good

Worth knowing

Clone a huge repository fast
`git clone --filter=blob:none <url>`
Full history, file contents on demand. Nothing is missing later.

Keep it fast
`git maintenance start`
Schedules the upkeep that keeps log and status quick.

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

Stash

A shelf for work that is not ready.

`git stash push -u -m "<label>"`
Set everything aside, untracked included

`git stash list`
What is shelved

`git stash pop`
Bring the newest back and drop it

`git stash apply stash@{<n>}`
Bring a specific one back, keep it shelved

`git stash show -p`
See inside before applying

Tags and releases

Permanent names on permanent commits.

`git tag -a v1.0.0 -m "<note>"`
Mark a release

`git push origin v1.0.0`
Tags do not push themselves

`git tag -d <tag>`
Delete locally; push origin --delete <tag> for the remote

`git describe --tags`
Nearest release to where you stand

`git switch --detach v1.0.0`
Visit a release; switch - to come back

Worth knowing

Label it or lose it
`git stash push -u -m "<what it was>"`
An unlabelled stash is a mystery by Thursday. The -u includes untracked files.

Look before you apply
`git stash show -p stash@{0}`
Shows the patch so nothing lands unseen.

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

History surgery

Everything here rewrites history. Fresh clone first, backup kept, force-with-lease after, teammates re-clone.

`git commit --fixup <hash>`
Mark a fix as belonging to an earlier commit

`git rebase -i --autosquash <hash>^`
Melt fixups into their targets

`git filter-repo --invert-paths --path <file>`
Erase a file from all history

`git filter-repo --strip-blobs-bigger-than 10M`
Erase every giant blob

`git push --force-with-lease`
The only force push worth typing

Submodules

A repository pinned inside another.

`git submodule add <url> <path>`
Pin another repository at an exact commit inside yours

`git clone --recurse-submodules <url>`
Clone a repo and its submodules in one step

`git submodule update --init --recursive`
Fill the empty folders after a plain clone

`git submodule update --remote`
Move pins to each submodule's latest commit

Worth knowing

Rewrite only what is yours
`git log --oneline origin/main..HEAD`
Anything listed here is unpushed and safe to reshape.

Never the shared branch
`git push --force-with-lease`
Refuses the push if the remote moved, which plain --force will not.

Clone one that has them
`git clone --recurse-submodules <url>`
A plain clone leaves the folders empty and confusing.

Or repair it afterwards
`git submodule update --init --recursive`
Fills them in without recloning.

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

How teams run Git

Working alone

Commit at every working state; push is your backup; branch before experiments

```
git add -A && git commit -m "<message>" && git push
```

Feature branch

The team default: main stays releasable, every change arrives by pull request

```
git switch -c <change> && git push -u origin <change>
```

Fork

For repositories you cannot push to, which is all of open source

```
gh repo fork <owner>/<repo> --clone
```

Trunk-based

Branches live hours, not weeks; tiny changes merge daily behind flags

```
git config --global pull.rebase true
```

Releases and hotfixes

A release is a tag; a hotfix is a branch from that tag, merged back after shipping

```
git switch -c hotfix-2.1.1 v2.1.0
```

The rules that keep teams safe. Never rewrite a branch others build on; rewrite your own freely with force-with-lease. Pull with rebase. Protect main so nothing lands without review and green checks. When anything is unclear: git status names the state you are in, and usually the way out.

Your first repository

`Name`
Short, lowercase, hyphens: weather-app

`Add a README`
The front page, and your first commit

`Add .gitignore`
Pick your language; noise stays out from day one

`Choose a license`
MIT if unsure; none means all rights reserved

The pull request path

branch · commit · push · open the PR · checks run · review answers with commits · merge, and the branch retires.

`gh pr create --fill`
Open it from the terminal

`gh pr checkout <number>`
Review one locally

`gh pr merge --squash --delete-branch`
Land it clean

The words nobody explains

`repository` a project and its entire history; everyone says repo

`README.md` the front page, rendered automatically

`Markdown` plain text with light marks; GitHub renders it everywhere

`.gitignore` what Git must never track

`LICENSE` the legal terms of reuse

`public, private` everyone reads, only you write; or invitation only

`star` a bookmark and a thank-you; stars rank repositories

`watch` subscribe to issues, PRs, and releases

`clone vs fork` to your machine; to your account

`issue` a tracked conversation about one bug or request

`default branch` usually main: what visitors see, what PRs target

`gist` one shareable file with its own URL

`organization` a shared account for a team

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

Practise on a graph you cannot break

A working Git engine runs inside the page at **amey-thakur.github.io/GIT-GUIDE/play.html**. Not a simulation of the output: commits, branches, HEAD, the index, file contents and the reflog are all modelled, so a merge really makes a second parent and a rebase really abandons the originals.

The two nobody lets you rehearse

A conflict with the markers still in it

`<<<<<<< HEAD`

The file opens for editing, both versions and all. Delete the markers, keep the version you want, and the sandbox refuses the commit while any remain.

The recovery drill, against a clock

`git reflog`

Three commits of work go in, and the sandbox destroys them with a hard reset. The Undo button is switched off, because on the day it happens to you there is no Undo. Bring them home and it tells you how long you took.

Undo anything

The situation, the command, and how much it costs. Green is safe, amber rewrites history, red can lose work.

Committed too early	`git reset --soft HEAD~1`	Safe
Staged the wrong file	`git restore --staged <file>`	Safe
Want the file back as committed	`git restore <file>`	Can lose work
Forgot a file in the last commit	`git add <file>`	Rewrites history
Wrong commit message	`git commit --amend -m "<new message>"`	Rewrites history
Amended and regretted it	`git reset --soft HEAD@{1}`	Safe
Merged the wrong branch	`git merge --abort`	Safe
Rebase went wrong	`git reset --hard ORIG_HEAD`	Can lose work
Pushed something you should not have	`git revert <hash>`	Safe
Pull brought in a mess	`git reset --hard ORIG_HEAD`	Can lose work
Need the project back at an old commit	`git switch --detach <hash>`	Safe

Undo anything, continued

One file should go back in time	`git restore --source=<hash> <file>`	Can lose work
A commit vanished	`git reflog`	Safe
Deleted a branch by mistake	`git branch <name> <hash>`	Safe
Committed on the wrong branch	`git branch <new-branch>`	Can lose work
A file must leave the last commit	`git reset HEAD~1 -- <file>`	Rewrites history
Need a clean tree for five minutes	`git stash push -u -m "<label>"`	Safe
Made a repository by accident	`rm -rf .git`	Can lose work
A file was deleted commits ago	`git log --diff-filter=D --oneline -- <path>`	Safe
Start over from what the remote has	`git fetch origin`	Can lose work
Reverted something you needed back	`git revert <revert-hash>`	Safe
Reset --hard took uncommitted work	`git fsck --lost-found`	Safe

I lost work: the order to try

Work down this list. Stop at the step that finds it.

1 **git status**

Name the state you are in. Half of all panics end here.

2 **git stash push -u**

Freeze everything before trying anything, so no fix can destroy more.

3 **git reflog**

Every position HEAD has held, kept about ninety days. Anything ever committed is here.

4 **git branch rescue <hash>**

Found it? Name it immediately. A named commit cannot be garbage collected.

5 **git fsck --lost-found**

For work that was staged but never committed: it survives as a dangling object.

6 **Editor local history**

Never staged, never committed? Git never saw it. Your editor may have.

The habit that makes this unnecessary. Commit early and often, and push daily. The reflog protects anything committed; nothing protects work that was never committed.

How much does this command cost

Every answer in this guide carries one of these three marks, and its own undo.

Safe

Nothing is lost, and most of it is reversible in a keystroke

`status` · `log` · `diff` · `fetch` · `switch` · `stash` · `revert` · `tag`

Rewrites history

The content survives, but commits change identity. Fine alone, rude on a shared branch

`rebase` · `commit --amend` · `reset --soft` · `filter-repo` · `push --force-with-lease`

Can lose work

Git may hold no copy afterwards. Rehearse first, and know the undo before you press enter

`reset --hard` · `clean -fd` · `push --force` · `branch -D` · `checkout -- <file>`

Rehearse first

`git clean -n -d`	before git clean -f -d, because cleaned files were never Git's to return
`git status`	before git reset --hard, so you know what is about to go
`git log --oneline HEAD..origin/main`	before any force push: anything listed is work you would erase

Two commands hold nothing back. git clean deletes files Git never tracked, and git reset --hard discards edits that were never committed. Everything else, the reflog can usually reach.

The errors you will actually meet

The message, the cause, the first command. All 500 decoded errors are searchable **on the site**.

`fatal: not a git repository (or any of the parent directories): .git`

This folder, and none above it, contains a .git directory. You are outside the project.

```
cd <project-folder>
```

`git@github.com: Permission denied (publickey).`

The host never received a key it recognizes: no key loaded, key not added to your account, or an HTTPS problem disguised by an SSH URL.

```
ssh -T git@github.com
```

`fatal: Authentication failed for 'https://github.com/...'`

The credential your machine sent is wrong, expired, or a password where a token is required.

```
gh auth login
```

`! [rejected] main -> main (non-fast-forward)`

The remote has commits you do not have. Git refuses to overwrite them.

```
git pull --rebase
```

`! [rejected] main -> main (fetch first)`

Same cause as non-fast-forward: someone pushed before you.

```
git pull --rebase
```

`CONFLICT (content): Merge conflict in <file>`

Both sides changed the same lines; Git needs a human decision.

```
git status
```

`fatal: Unable to create '.git/index.lock': File exists.`

Another Git process is running, or one crashed and left its lock behind.

```
rm -f .git/index.lock
```

`You are in 'detached HEAD' state.`

Not an error. You checked out a commit or tag directly, so you stand on a snapshot instead of a branch.

```
git switch -c <name>
```

`Your branch and 'origin/main' have diverged`

Both you and the remote gained different commits since you last synced.

```
git pull --rebase
```

`error: src refspec main does not match any`

The branch you are pushing does not exist locally, almost always because there is no commit yet.

```
git status
```

Errors, continued

`fatal: refusing to merge unrelated histories`

The two sides share no common commit, typically a fresh git init pulled against a remote that already has commits.

```
git pull origin main --allow-unrelated-histories
```

`fatal: detected dubious ownership in repository`

The repository folder belongs to a different OS user than the one running Git, common in containers, WSL, and network drives.

```
git config --global --add safe.directory <path>
```

`remote: error: File <name> is <size> MB; this exceeds GitHub's file size limit of 100.00 MB`

One specific file crossed the hard limit; the push cannot land while any commit carries it.

```
git rm --cached <big-file>
```

`fatal: could not read Username for 'https://github.com': No such device or address`

Git wanted to ask for credentials but no terminal was attached, the classic CI and script failure.

```
git remote set-url origin https://x-access-
token:${TOKEN}@github.com/<owner>/<repo>.git
```

`fatal: The current branch has no upstream branch`

This local branch has never been connected to a remote branch, so plain git push does not know where to go.

```
git push -u origin <branch>
```

`nothing to commit, working tree clean`

Commit found no staged changes. Either everything is already committed, or the edits live elsewhere.

```
git status
```

`fatal: the remote end hung up unexpectedly`

The connection died mid-transfer, and Git may report it as early EOF. Usually a large clone or push meeting a proxy, server, or network limit.

```
git config http.postBuffer 157286400
```

`remote: Repository not found.`

The URL has a typo, the repository is private and you lack access, or you are authenticated as the wrong account.

```
git remote -v
```

`Host key verification failed.`

SSH does not yet trust this server's fingerprint, or the stored one no longer matches.

```
ssh -T git@github.com
```

`warning: LF will be replaced by CRLF the next time Git touches it`

Windows line endings meeting Unix line endings. Harmless, but configure it once and it goes quiet.

```
printf '* text=auto\n' >> .gitattributes
```

The living version

Nine doors, one address: **amey-thakur.github.io/GIT-GUIDE**

Finder
ask anything, or paste the error

Start
zero to first push

Learn
the model, step by step

Practise
a real Git graph to play on

Fix
guided rescue

Errors
Git's messages, decoded

GitHub
first repo to branch protection

Workflows
how teams run Git

Cheatsheet
everything, one line each

SAFE REWRITES HISTORY DESTRUCTIVE

every answer carries its danger level and its undo

Every answer also lives in the repository's Discussions; a question the guide cannot answer becomes its next addition. MIT licensed, no trackers, works offline.

Amey Thakur
amey-thakur.github.io/GIT-GUIDE